

Graph Preconditioning for High-Dimensional Fixed Effects Regression

Alexander Fischer¹ and Kristof Schröder²

Draft: June 2026

Abstract. The Method of Alternating Projections (MAP) is the de facto standard algorithm for estimating high-dimensional fixed-effect regressions. Although MAP is often fast, it can converge slowly when the fixed-effect structure is poorly connected, as in matched employer-employee panels with low worker mobility between firms. We relate MAP’s convergence to the connectivity of the weighted bipartite graph formed by the levels of a pair of fixed-effect dimensions and propose a novel graph-preconditioned Krylov solver. Our preconditioner is constructed from sparse approximate Cholesky factorizations of the weighted bipartite graph Laplacian associated with each pair of fixed-effect dimensions. We show that graph preconditioning can substantially improve convergence on poorly connected fixed-effect graphs but that its setup time can dominate total runtime on well-connected graphs. On a near-nested design with 10 million observations, factor-pair LSMR completes in 4.63 seconds, compared with 62.2 seconds for the fastest MAP implementation.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed-effect regressions are ubiquitous in applied econometrics with roughly half of published research in top economics and finance journals mentioning “fixed effects” (Goldsmith-Pinkham 2026). Labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia; health economists study physician practice styles with individual-physician and region fixed effects in mover designs; and education researchers study models with school, student, teacher, or student-teacher fixed effects.

The standard computational starting point for estimating these regressions efficiently is the Frisch-Waugh-Lovell (FWL) theorem (Frisch and Waugh 1933; Lovell 1963). FWL reduces the fixed-effect estimation problem to “residualizing” dependent and independent variables against the fixed effects, and then running a low-dimensional regression on the residualized variables.

The workhorse method for these fixed-effect residualizations is the Method of Alternating Projections (MAP), also known as iterative demeaning or the “Zig-Zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013). Most leading software implementations of fixed-effect regression, such as `reghdfe` in Stata

¹trivago

²appliedAI Institute for Europe GmbH

(Correia 2014; 2017), `fixest` (Bergé et al. 2026) in R, or `PyFixest` in Python (PyFixest Developers 2026), use methods based on iterative demeaning, often with acceleration.

MAP cycles over the fixed-effect dimensions and, for each variable of interest, subtracts the mean within every level of the current dimension. Because demeaning along one fixed-effect dimension (for example workers) changes the group means for the other dimensions (for example firms or years), the procedure repeats these cycles until convergence. In other words, MAP residualizes with respect to one fixed-effect dimension at a time but does not directly exploit the co-occurrence of levels of different fixed-effect dimensions. In a worker-firm panel, for example, low worker mobility between firms can make some directions in the worker-firm fixed-effect structure nearly collinear, adversely affecting MAP’s convergence.

We show that this problem is related to the connectivity of the graph formed by the fixed effects: The levels of a pair of fixed-effect dimensions, for example workers and firms, are the nodes of a bipartite graph where two nodes are connected if there is an observation taking on the respective fixed-effect levels (for example, a worker working at a particular firm). The bipartite graph is weighted by the number of occurrences of a given pair of fixed-effect levels.

We follow Jochmans and Weidner (2019) and use the second-smallest eigenvalue of the normalized Laplacian as our measure of connectivity. For factor pair (q, r) , let L_{qr} be the weighted graph Laplacian and Δ_{qr} its degree matrix. Then

$$S_{qr} = \Delta_{qr}^{-\frac{1}{2}} L_{qr} \Delta_{qr}^{-\frac{1}{2}}. \quad (1)$$

Our measure of connectivity for the factor pair is $\lambda_2(S_{qr})$. Edge weights are observed co-occurrence counts, and a larger value indicates stronger connectivity between the two fixed-effect dimensions. Our empirical results report our measure of connectivity for the worker-firm graph.³

In this paper, we use these pairwise bipartite graphs to construct an additive Schwarz preconditioner for LSMR (Fong and Saunders 2011), a Krylov least-squares solver. The preconditioner provides an inexpensive approximation to the inverse of the fixed-effect Gramian, improving the conditioning of the Krylov iteration without changing the least-squares solution.⁴ Algebraically, each off-diagonal block of the fixed-effect Gramian records the edge weights of one pairwise bipartite graph. When combined with the corresponding diagonal count blocks and subjected to a sign flip for one fixed-effect dimension, the resulting pair block is a graph Laplacian (Correia 2017). We use sparse approximate Cholesky factorizations (Spielman and Teng 2014; Gao et al. 2025) to obtain efficient approximate solves for these pairwise Laplacians and combine their corrections in the Schwarz preconditioner.

In our benchmarks, graph preconditioning is substantially faster than MAP on poorly connected designs. On well-connected designs, however, its setup time can dominate total runtime in which case MAP and diagonally preconditioned Krylov methods are faster.

³We compute our measure of connectivity after iterative singleton removal. If the remaining graph is disconnected, we use the component containing the most retained observations and report its observation share; Section 7 gives the full convention.

⁴The Julia implementation of fixed-effect regression, `FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026), uses the same Krylov solver but only with diagonal preconditioning. Diagonal preconditioning ignores the off-diagonal co-occurrence structure; our contribution is the preconditioner, not the use of LSMR.

The pair factorizations have a setup cost, so they are unlikely to help when MAP already converges in a few passes. On a weakly connected graph, however, they can save substantial iteration time. Figure 1 reports total regression time as mobility and sorting change: the left column compares package defaults, whereas the right holds the package and achieved accuracy fixed within PyFixest using tolerances calibrated in Section 7.

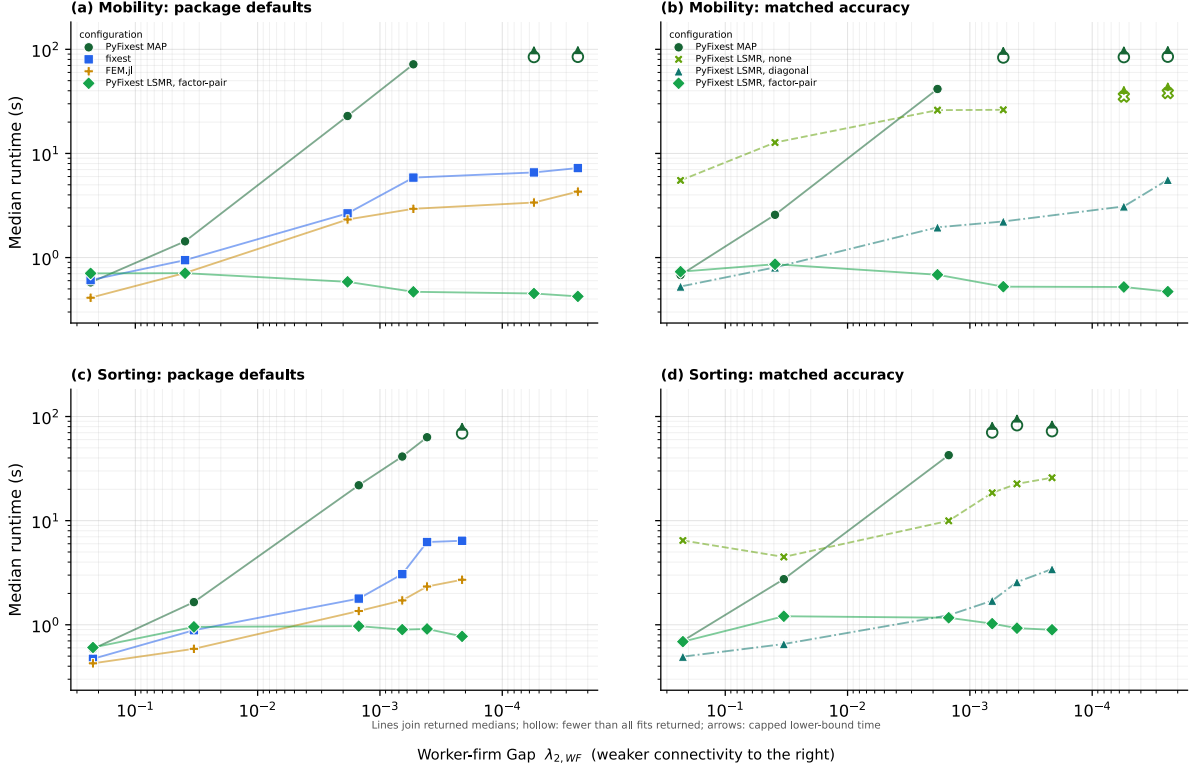


Figure 1: Median regression time against our measure of connectivity for the worker-firm graph, $\lambda_{2,WF}$, on logarithmic axes. The measure is computed after singleton removal on the connected component containing the most observations; larger values mean stronger connectivity, and the shared reversed horizontal scale makes connectivity weaken from left to right. Each simulated worker-firm-year panel has 1 million observations, with worker mobility varying in the top row and sorting among movers in the bottom row. The left column compares package defaults, including each package’s singleton rule, while the right compares four PyFixest configurations at tolerances that give comparable coefficient and residual errors in Section 7. A filled marker means all three planned fits returned estimates; a hollow marker means one or two did. Lines join observed medians rather than fitted trends, and an arrow marks the median time at the iteration cap, a lower bound on completion time; other failures are omitted.

All implementations finish quickly when our measure of connectivity is high. MAP and unpreconditioned LSMR slow sharply as the measure falls; factor-pair LSMR remains fast, and its runtime falls in the lowest-mobility designs because the local graphs become cheaper to factorize.

The rest of the paper is organized as follows. Section 2 sets up the fixed-effect absorption problem, and Section 3 introduces the AKM (Abowd et al. 1999) model as our running example. Section 4 develops the graph structure of the fixed-effect Gramian, and Section 5 connects this structure to the convergence behavior of MAP. Section 6 builds up the graph preconditioner, starting from a general discussion of preconditioning and culminating in the construction of the graph-based preconditioner. Section 7 reports

benchmarks on runtime, memory, and numerical equivalence; Section 8 describes the software implementation of the new algorithm; and Section 9 concludes.

2. Absorbing Fixed Effects⁵

We focus on the linear model

$$y = X\beta + D\alpha + \varepsilon, \quad (2)$$

where X denotes a set of covariates and D is the fixed-effect design matrix given by

$$D_{ij} = \begin{cases} 1 & \text{if observation } i \text{ belongs to fixed-effect level } j, \\ 0 & \text{otherwise.} \end{cases} \quad (3)$$

In high-dimensional applications, D may have hundreds of thousands or millions of columns, and forming or inverting the full system in $[X \ D]$ might prove computationally infeasible.

Fortunately, the Frisch-Waugh-Lovell (FWL) theorem (Frisch and Waugh 1933; Lovell 1963) allows us to estimate β in two tractable steps without forming $[X \ D]$ or inverting its cross product. First, the outcome and each covariate are residualized against the fixed effects by regressing y and each column of X on D . Second, the residualized outcome $\tilde{y}$ is regressed on the residualized covariates $\tilde{X}$. The FWL theorem implies that the resulting slope of this regression equals the coefficient β of the full model Equation 2.

To fix notation, we denote by M_D the linear operator that maps a variable to its residual when regressed on the fixed effects D , i.e., $\tilde{y} = M_D y$ and $\tilde{X} = M_D X$. The coefficient of interest β is then recovered by regressing the residualized outcome on the residualized regressors

$$\tilde{y} = M_D y, \quad \tilde{X} = M_D X, \quad \hat{\beta} = (\tilde{X}' W \tilde{X})^{-1} \tilde{X}' W \tilde{y}, \quad (4)$$

where W is a diagonal matrix of weights. The residualization step is the weighted least-squares projection of the outcome and each covariate in X onto the column space of the fixed-effect design matrix D . For a right-hand side μ , i.e., the outcome y or a column of X , we solve

$$\hat{\alpha}_\mu \in \arg \min_{\alpha} \| D\alpha - \mu \|_W^2, \quad (5)$$

The first-order condition for Equation 5, $D'W(D\hat{\alpha}_\mu - \mu) = 0$, can be written as

$$G\hat{\alpha}_\mu = D'W\mu, \quad G = D'WD, \quad (6)$$

where G is the fixed-effect Gramian. Solving Equation 6 for $\hat{\alpha}_\mu$, we obtain the residualized right-hand side as $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$.⁶ We note that each FWL residualization uses the same fixed-effect Gramian G , and only the right-hand side μ changes as we move from the outcome to the covariates. The cost of residualization therefore depends on the structure of the Gramian G .

We illustrate this structure with the AKM worker-firm model in the next section before explaining the graph interpretation of the fixed-effect Gramian G in Section 4 and how it governs MAP convergence in Section 5.

⁵Researchers employ several names for this operation: “absorbing fixed effects”, “demeaning”, “residualizing”, or applying the “within transformation”. We use these terms interchangeably throughout.

⁶With a full set of fixed-effect dummies, the columns of D are linearly dependent and $\hat{\alpha}_\mu$ is not unique. To report individual fixed effects, one must choose a normalization, usually by dropping reference categories. The normalization does not change the fitted value, the residual, or the FWL slope. Throughout, inverses are taken after normalizing each connected component.

3. A Running Example: The AKM Model

One of the most prominent examples of high-dimensional fixed-effect regressions is the Abowd-Kramarz-Margolis (AKM) model (Abowd et al. 1999). The AKM model separates persistent worker heterogeneity from firm wage premia using workers who move across firms. We write the AKM regression equation as

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x'_{it}\beta + \varepsilon_{it}, \quad (7)$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t , and φ_t is a time fixed effect.

The AKM specification has a natural graph representation. Workers and firms are nodes in a bipartite graph, with an edge connecting a worker to the firm they are employed by. Each connection is weighted by the count of observations of a given worker-firm pair across years. Stayers—workers who never change their employer—add weight to existing worker-firm links but do not connect different firms. Figure 2 contrasts a well-connected mobility graph with one that fragments under strong sorting.

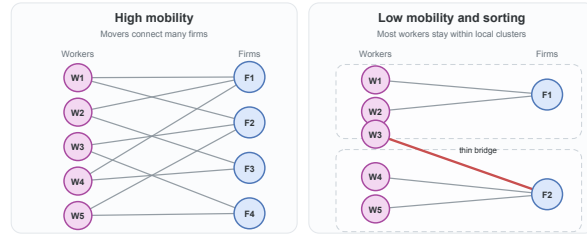


Figure 2: Worker-firm graph connectivity. When mobility is high, many paths connect firms. With low mobility and strong sorting, the graph breaks into nearly separate clusters joined only by narrow bridges.

Worker mobility determines the graph’s connectedness and enables separate identification of worker and firm effects (Bonhomme et al. 2023). For a worker observed at only one firm, a high wage could reflect an unusually productive worker, a high-wage firm, or both. A worker earning high wages across multiple firms provides evidence of a worker effect, while different workers earning high wages at the same firm provide evidence of a firm premium. The more cross-firm comparisons the data contain, the easier it becomes to separately identify worker effects and firm premia. This is why the identifying variation generated by movers corresponds to the connectedness of the fixed-effect graph.

4. The Graph Structure of the Gramian

The bipartite graph of worker and firm connections introduced in Section 3 is represented algebraically in the block structure of the Gramian $G = D'WD$ derived in Equation 6 (Correia 2017). Suppose that the columns of D are ordered as worker levels, firm levels, and year levels. Then, the Gramian has the block structure

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \quad (8)$$

The **diagonal blocks** G_{WW} , G_{FF} , and G_{YY} contain the weighted counts for each worker, firm, and year, respectively. Because one observation belongs to exactly one level of each fixed-effect dimension, these blocks are diagonal and solving them only requires division by weighted group counts. The **off-diagonal**

blocks are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j with analogous interpretations for the worker-year block C_{WY} and the firm-year block C_{FY} .

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0
6	W_3	F_2	Y_2	4.5

Table 1: Synthetic worker-firm panel

As an instructive example, we construct a small synthetic worker-firm panel in Table 1 and populate its Gramian G . Throughout the example, we assume an unweighted regression with $W = I$. Figure 3 illustrates the bipartite worker-firm graph of this panel. Worker W_1 is observed at two firms F_1 and F_2 , creating a link between them, while workers W_2 and W_3 are each observed at a single firm. In AKM terms, W_1 is a mover and W_2 and W_3 are stayers.

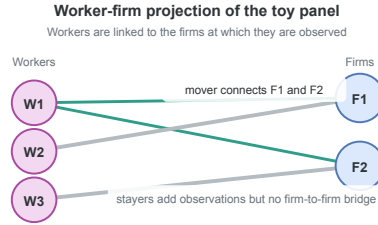


Figure 3: Worker-firm projection of the example panel. Worker W_1 is a mover; workers W_2 and W_3 are stayers.

Because the regression is unweighted, $W = I$, the diagonal blocks of the Gramian in Equation 8 are simple counts. Table 1 shows that each worker is observed twice, each firm three times, and each year three times, so that

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \quad (9)$$

The off-diagonal blocks are cross-tabulations between fixed-effect dimensions. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \quad (10)$$

The first row indicates that worker W_1 appears once at firm F_1 and once at firm F_2 . Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2 , respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \quad (11)$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \quad (12)$$

Firm F_1 appears twice in year Y_1 and once in year Y_2 , and firm F_2 has the opposite pattern. With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full Gramian is

$$G = \begin{pmatrix} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{pmatrix}. \quad (13)$$

The submatrix comprising the worker-worker, worker-firm, and firm-firm blocks algebraically represents the bipartite graph formed by the worker and firm fixed effects. The diagonal entries are worker and firm counts, and the entries of C_{WF} are edge multiplicities between workers and firms. After flipping the sign of C_{WF} , the worker-firm submatrix is the graph Laplacian of the bipartite graph

$$L_{WF} = \begin{pmatrix} G_{WW} & -C_{WF} \\ -C_{(WF)'} & G_{FF} \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{pmatrix}. \quad (14)$$

The off-diagonal entries of L_{WF} are non-positive, and every row sums to zero because each diagonal count cancels the off-diagonal observation counts in the same row. For a worker, the diagonal entry is the number of observations for that worker, while the off-diagonal entries show how those observations are distributed across firms. Firm rows have the analogous interpretation with observations summed over workers.

The graph Laplacian can be constructed similarly for any pair of fixed-effect dimensions which we will use to construct a preconditioner in Section 6. First, however, we discuss the method of alternating projections, which avoids forming the full Gramian G and only uses the diagonal worker, firm, and year blocks.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for high-dimensional fixed-effect regressions is the Method of Alternating Projections (MAP), also referred to as iterative demeaning or the “zig-zag” algorithm (Guimaraes and Portugal 2010; Gaure 2013). Many packages employ MAP or its variants, frequently combined with accelerations (Berge 2018; Correia 2017), such as the Irons-Tuck extrapolation used by `fixest` (Irons and Tuck 1969; Bergé et al. 2026). MAP solves the FWL residualization problem by iterating over one fixed-effect dimension at a time. For example, in the AKM model of Section 3, MAP first subtracts worker means from the current residual, then firm means from the updated residual, then year means, and repeats until convergence is reached.

To relate MAP’s convergence to the connectivity of the fixed-effect graph, we discuss the FWL normal equations presented in Equation 6. We start with the case of the AKM model where the fixed-effect design matrix becomes $D = [D_W \quad D_F \quad D_Y]$ for worker, firm, and year fixed effects. Equation 6 can then be written as

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'}W\mu \\ D_{F'}W\mu \\ D_{Y'}W\mu \end{pmatrix}, \quad (15)$$

or, equivalently,

$$\begin{aligned} G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y &= D_{W'}W\mu, \\ C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y &= D_{F'}W\mu, \\ C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y &= D_{Y'}W\mu. \end{aligned} \quad (16)$$

Each equation can be rearranged to express one block of effects conditional on the others. For example, using $C_{WF} = D_{W'}WD_F$ and $C_{WY} = D_{W'}WD_Y$, we can substitute $D_{W'}W$ in Equation 16 and obtain

$$\alpha_W = G_{WW}^{-1}D_{W'}W(\mu - D_F\alpha_F - D_Y\alpha_Y). \quad (17)$$

Because G_{WW} is a diagonal matrix whose entries are workers' total observation weights, solving for α_W divides each worker's weighted partial residual by its total observation weight.

For a general fixed-effect model with $q = 1, \dots, Q$ fixed-effect dimensions, MAP updates the coefficient vector $\alpha_q^{(k+1)}$ of fixed-effect dimension q at iteration $k + 1$ using the Gauss-Seidel algorithm (Guimaraes and Portugal 2010) according to

$$\alpha_q^{(k+1)} = G_{qq}^{-1}D_{q'}W \left(\mu - \sum_{s=1}^{q-1} D_s\alpha_s^{(k+1)} - \sum_{s=q+1}^Q D_s\alpha_s^{(k)} \right). \quad (18)$$

Note that the cross-tabulation blocks $C_{qs} = D_{q'}WD_s$ are never used directly in Equation 18 and enter in MAP only indirectly through the propagation of the error across the different fixed-effect dimensions. To see this, let $\hat{\alpha} = (\hat{\alpha}_1, \dots, \hat{\alpha}_Q)$ be a solution to Equation 6 and define the coefficient error at iteration k as $e_q^{(k)} = \alpha_q^{(k)} - \hat{\alpha}_q$. Subtracting the solution $\hat{\alpha}$ from Equation 18 yields

$$e_q^{(k+1)} = -G_{qq}^{-1} \left(\sum_{s=1}^{q-1} C_{qs}e_s^{(k+1)} + \sum_{s=q+1}^Q C_{qs}e_s^{(k)} \right). \quad (19)$$

Define the degree-scaled errors $\tilde{e}_q^{(k)} = G_{qq}^{\frac{1}{2}}e_q^{(k)}$ and the degree-normalized cross-tabulations

$$H_{qs} = G_{qq}^{-\frac{1}{2}}C_{qs}G_{ss}^{-\frac{1}{2}}. \quad (20)$$

Then, Equation 19 can be written as

$$\tilde{e}_q^{(k+1)} = - \sum_{s=1}^{q-1} H_{qs}\tilde{e}_s^{(k+1)} - \sum_{s=q+1}^Q H_{qs}\tilde{e}_s^{(k)}. \quad (21)$$

The degree-normalized cross-tabulations H_{qs} (20) appear as the off-diagonal blocks of the normalized graph Laplacian

$$\begin{aligned}
\mathcal{L}_{qs} &= \begin{pmatrix} G_{qq}^{-\frac{1}{2}} & 0 \\ 0 & G_{ss}^{-\frac{1}{2}} \end{pmatrix} L_{qs} \begin{pmatrix} G_{qq}^{-\frac{1}{2}} & 0 \\ 0 & G_{ss}^{-\frac{1}{2}} \end{pmatrix} \\
&= \begin{pmatrix} G_{qq}^{-\frac{1}{2}} & 0 \\ 0 & G_{ss}^{-\frac{1}{2}} \end{pmatrix} \begin{pmatrix} G_{qq} & -C_{qs} \\ -C_{(qs)'} & G_{ss} \end{pmatrix} \begin{pmatrix} G_{qq}^{-\frac{1}{2}} & 0 \\ 0 & G_{ss}^{-\frac{1}{2}} \end{pmatrix} \\
&= \begin{pmatrix} I & -H_{qs} \\ -H_{(qs)'} & I \end{pmatrix},
\end{aligned} \tag{22}$$

where L_{qs} is the graph Laplacian of the bipartite graph formed by the fixed-effect dimensions q and s .

The connectivity of the bipartite graph formed by a pair of fixed-effect dimensions therefore governs the speed of convergence of MAP. To see this, note that the second-smallest eigenvalue of the normalized graph Laplacian $\lambda_2(\mathcal{L}_{qs})$ is our measure of connectivity⁷ and that, within each connected component, the block structure in Equation 22 implies that $\lambda_2(\mathcal{L}_{qs})$ is related to the second-largest singular value of the degree-normalized cross-tabulation $\sigma_2(H_{qs})$ via

$$\lambda_2(\mathcal{L}_{qs}) = 1 - \sigma_2(H_{qs}). \tag{23}$$

The largest nontrivial singular value of the degree-normalized cross tabulation $\sigma_2(H_{qs})$ provides a measure of worst-case convergence of MAP. Indeed, for a model containing only two fixed-effect dimensions q and s , the MAP error update (19) becomes

$$\begin{aligned}
\tilde{e}_q^{(k+1)} &= -H_{qs} \tilde{e}_s^{(k)}, \\
\tilde{e}_s^{(k+1)} &= H_{(qs)'} H_{qs} \tilde{e}_s^{(k)}.
\end{aligned} \tag{24}$$

Consequently, the error component of $\tilde{e}_s^{(k+1)}$ in the direction of the singular vector associated with $\sigma_2(H_{qs})$ shrinks with the worst-case MAP contraction factor

$$\rho_{qs}^{\text{MAP}} = \sigma_2(H_{qs})^2 = (1 - \lambda_2(\mathcal{L}_{qs}))^2. \tag{25}$$

In other words, a poorly connected graph implies that $\lambda_2(\mathcal{L}_{qs})$ is small and $\sigma_2(H_{qs})$ is close to one, so ρ_{qs}^{MAP} is close to one and the error component potentially shrinks slowly in each MAP iteration. With more than two fixed-effect dimensions, the full error propagation also depends on the remaining dimensions and their update order, so our pairwise measure of connectivity does not fully describe MAP's convergence rate.

For the worker-firm example discussed in Section 4, the diagonal blocks are $G_{WW} = \text{diag}(2, 2, 2)$, $G_{FF} = \text{diag}(3, 3)$, and the cross-tabulation is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}, \tag{26}$$

so that

⁷it is zero if the graph is disconnected and is small when the graph consists of nearly disconnected subgraphs joined only by narrow bridges (Chung 1997)

$$H_{WF} = G_{WW}^{-\frac{1}{2}} C_{WF} G_{FF}^{-\frac{1}{2}} = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \quad (27)$$

The graph is connected, and $H_{(WF)'} H_{WF} = \frac{1}{6} \begin{pmatrix} 5 & 1 \\ 1 & 5 \end{pmatrix}$ has eigenvalues 1 and $\frac{2}{3}$, hence $\sigma_2(H_{WF}) = \sqrt{\frac{2}{3}}$ and our measure of connectivity is $\lambda_2(\mathcal{L}_{WF}) = 1 - \sqrt{\frac{2}{3}}$, while the worst-case MAP contraction factor is $\rho_{WF}^{\text{MAP}} = \frac{2}{3}$.

6. The Factor-Pair Schwarz Preconditioner

In the previous Section 5, we relate MAP's convergence to the connectivity of the pairwise fixed-effect graphs. Our goal is to incorporate information on the graph's connectivity to improve convergence in poorly connected fixed-effect graphs. To this end, we use the pairwise fixed-effect graphs to construct a preconditioner for LSMR (Fong and Saunders 2011), an iterative least-squares algorithm that improves an initial guess through repeated residual corrections.

6.1. Preconditioners

LSMR's convergence depends on the conditioning of the fixed-effect Gramian G . To see this, recall from Equation 6 that the fixed-effect coefficients satisfy $G\alpha = D'W\mu$. For a candidate solution α_k at iteration k , the residual is

$$s_k = D'W\mu - G\alpha_k = G(\hat{\alpha} - \alpha_k). \quad (28)$$

Consequently, components of the residual s_k along eigenvector directions of the Gramian G with small eigenvalue contribute little to the residual even when their coefficient error is large. Note that because, up to a sign change, the pairwise Gramian is the pairwise graph Laplacian discussed in Section 5, these eigenvector directions correspond to the poorly connected bipartite fixed-effect graphs. This means that LSMR removes some error components after a small number of iterations, while components corresponding to weak links, sparse mobility, or near nesting in the fixed-effect graph shrink more slowly.

A preconditioner M^{-1} is designed to counteract this imbalance by approximating the inverse of the Gramian G^{-1} , so that $M^{-1}s_k$ yields an approximation of the error $\hat{\alpha} - \alpha_k$. If $M^{-1} = G^{-1}$, the preconditioned operator is $M^{-1}G = G^{-1}G = I$, in which case the solver would recover the solution after one correction. However, to form G^{-1} one would have to solve the fixed-effect normal equations themselves which is computationally expensive. For a preconditioner to be useful, it must therefore approximate G^{-1} cheaply so that repeated preconditioning amortizes its setup cost.⁸

6.1.1. The Diagonal Preconditioner

The simplest approximation to G^{-1} discards the off-diagonal cross-tabulations of the fixed-effect Gramian to construct a diagonal preconditioner (Xu 1992).⁹ In the AKM model, the diagonal preconditioner of the Gramian (8) takes the form

$$M_{\text{diag}}^{-1} = \text{diag}(G_{WW}^{-1}, G_{FF}^{-1}, G_{YY}^{-1}). \quad (29)$$

⁸LSMR never forms $M^{-1}G$ explicitly, nor G itself. The iteration requires only products with D and D' and applications of M^{-1} , supplied as linear operators.

⁹The diagonal preconditioner is used by `FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026)

It encodes the number of observations of each fixed-effect level, but discards the connectivity of the bipartite fixed-effect graph and therefore does not address potentially slow convergence due to poorly connected bipartite fixed-effect graphs.

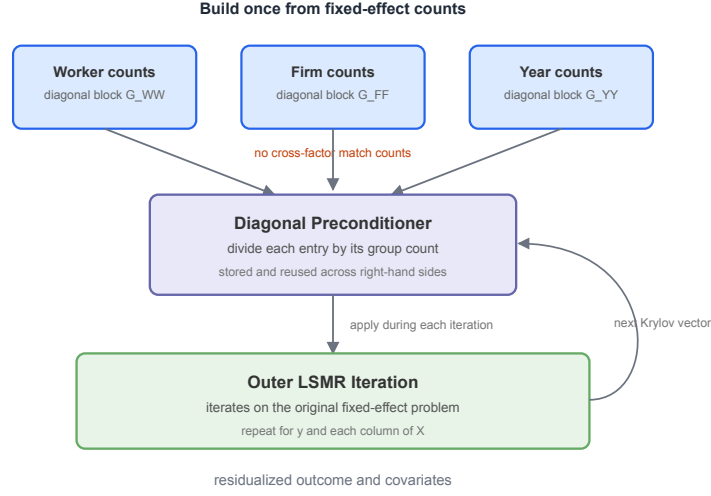


Figure 4: Diagonal preconditioning within LSMR. Worker, firm, and year counts are computed once and reused for the outcome and covariates; at each iteration, the preconditioner divides factor-level entries by their total observation weights. Because LSMR continues to check the original fixed-effect least-squares problem, diagonal scaling affects convergence speed rather than the residualized variables reached at convergence.

6.1.2. The Factor-Pair Schwarz Preconditioner

To incorporate information on the pairwise graph connectivity, we use additive Schwarz preconditioning (Toselli and Widlund 2005) and approximate G^{-1} as the sum of inverses of the pairwise fixed-effect Gramians

$$G_{qr} = R_{qr} G R_{(qr)'} = \begin{pmatrix} G_{qq} & C_{qr} \\ C_{(qr)'}' & G_{rr} \end{pmatrix}, \quad (30)$$

where R_{qr} restricts the coefficient space to the coordinates of the fixed-effect dimensions (q, r) . Because the pairwise coefficient spaces overlap, we combine their inverse contributions using partition-of-unity weights. For each fixed-effect level j in dimension q or r , let c_j denote the number of pairwise fixed-effect Gramians containing j and let Ω_{qr} be the diagonal matrix with entries $c_j^{-\frac{1}{2}}$. The additive Schwarz preconditioner is then the sum of local factor-pair inverses

$$P^{-1} = \sum_{q < r} P_{qr}^{-1} = \sum_{q < r} R_{(qr)'} \Omega_{qr} G_{qr}^{-1} \Omega_{qr} R_{qr}. \quad (31)$$

Applied to the AKM model and its Gramian (8), the additive Schwarz preconditioner is given by

$$P^{-1} = P_{WF}^{-1} + P_{WY}^{-1} + P_{FY}^{-1}. \quad (32)$$

The Schwarz preconditioner depends on the three pairwise fixed-effect Gramians G_{WF} , G_{WY} and G_{FY} , and consequently, incorporates the information on the connectivity of the bipartite fixed-effect graphs encoded in the cross-tabulations C_{WF} , C_{WY} and C_{FY} . For the worker-firm pair, for example, we have

$$P_{WF}^{-1} = \frac{1}{2} R_{(WF)'} \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'}' & G_{FF} \end{pmatrix}^{-1} R_{WF}. \quad (33)$$

Figure 5 illustrates that the additive Schwarz preconditioner incorporates the pairwise cross-tabulations C_{WF} in contrast to MAP (Section 5) and the diagonal preconditioner (Section 6.1.1) which only solve the diagonal block G_{WW} . The Schwarz preconditioner does not solve the full simultaneous worker-firm-year problem but only the local factor-pair subproblems. However, even the factor-pair subproblems may contain hundreds of thousands of levels per fixed-effect dimension, making exact inversion computationally expensive. In the next section, we discuss efficient approximate solutions to the local factor-pair subproblems.

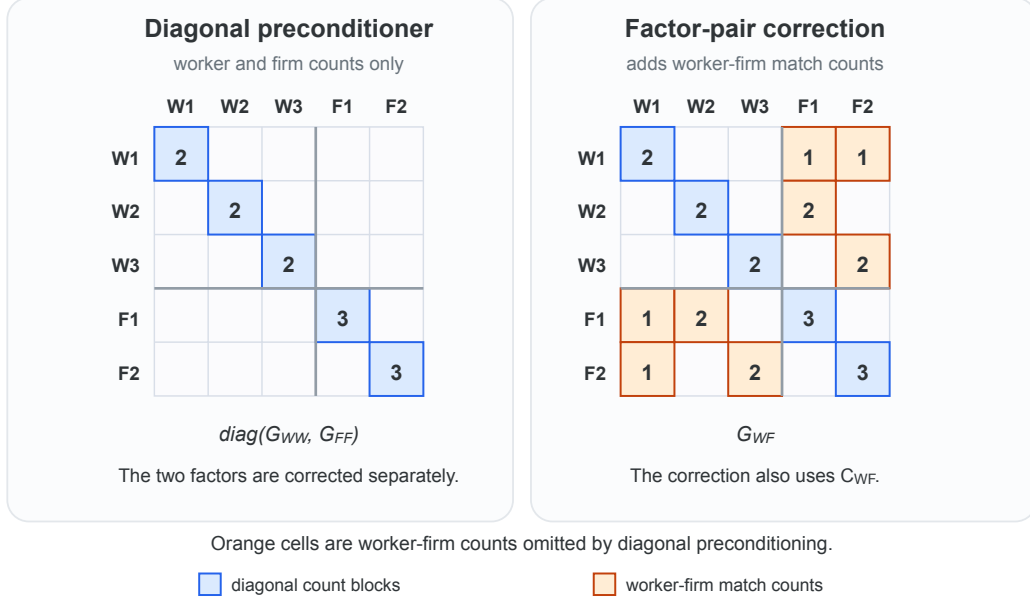


Figure 5: Count-only and factor-pair corrections for the example from Section 4. The diagonal preconditioner (left) treats the blue worker and firm count blocks separately, whereas the factor-pair correction (right) retains those blocks and adds the orange worker-firm observation counts C_{WF} .

6.2. Approximate Factor-Pair Solves via Graph Laplacians

Applying the exact factor-pair inverses P_{qr}^{-1} in Equation 31 can become computationally expensive when the number of fixed-effect levels is large. For preconditioning, however, this local solve need not be exact. Recognizing that the pairwise fixed-effect Gramians are, up to a sign change, the graph Laplacian of the bipartite fixed-effect graph, we can efficiently construct approximate solutions to the local subproblems.

For two fixed-effect dimensions (q, r) , the graph Laplacian L_{qr} is given by

$$L_{qr} = \begin{pmatrix} G_{qq} & -C_{qr} \\ -C_{(qr)'} & G_{rr} \end{pmatrix} = T_{qr} G_{qr} T_{qr}, \quad (34)$$

where $T_{qr} = \text{diag}(I_q, -I_r)$ for identity matrices I_q and I_r . Because the graph Laplacian L_{qr} admits sparse approximate Cholesky factorizations (Gao et al. 2025), we can efficiently construct approximate solutions to

$$G_{qr}^{-1} = T_{qr} L_{qr}^{-1} T_{qr}. \quad (35)$$

The approximate Cholesky factorization has expected construction cost $\mathcal{O}(m \log m)$, where m denotes the number of pairwise links (Gao et al. 2025). Let A_{qr} denote the resulting operator approximating G_{qr}^{-1} . Then, the additive Schwarz preconditioner becomes

$$M^{-1} = \sum_{q < r} R_{(qr)'} \Omega_{qr} A_{qr} \Omega_{qr} R_{qr}. \quad (36)$$

Our preconditioner therefore makes two approximations: First, the inverse of the fixed-effect Gramian G^{-1} is approximated by a sum of factor-pair inverses P^{-1} . Second, the inverse of the pairwise fixed-effect Gramians G_{qr}^{-1} is replaced with the approximate solution A_{qr} .

6.3. Implementation

Figure 6 shows the construction. For each pair of absorbed factors, a sign change converts the local Gramian to a graph Laplacian. The implementation handles large connected components with approximate Schur reduction and sparse approximate Cholesky, then maps the corrections back to the full coefficient vector and combines them using partition-of-unity weights.

The outer LSMR iteration uses this preconditioner to shape its search directions (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2025). Once built, the same preconditioner can residualize the outcome and every covariate. Appendix A lists the setup and application steps.

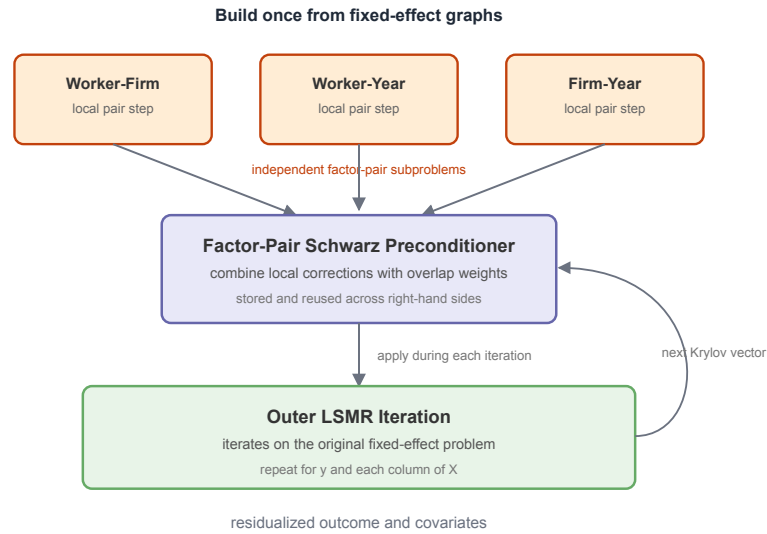


Figure 6: Construction of the factor-pair preconditioner. For large pair blocks, the algorithm approximates the Schur complement and factors the reduced Laplacian with approximate Cholesky. The weighted pair corrections form the preconditioner used by LSMR.

7. Benchmarks

The benchmarks ask when fewer iterations repay the cost of building the factor-pair preconditioner. We vary mobility and sorting in controlled AKM panels, compare two public synthetic designs, and use eight real datasets from Correia (2017).

Elapsed times cover the package-level regression call, from model setup through coefficient estimation. Cross-package comparisons use the same input data but retain each package’s default treatment of

singleton levels and separated observations, whereas experiments that isolate the preconditioner use one prepared sample. The appendix reports the additional memory used to store the pair structures.

Each table reports our measure of connectivity for each factor pair after iterative singleton removal. For a disconnected graph, the value comes from the component with the most retained observations, whose observation share appears in parentheses. As in Section 5, the measure remains a pairwise diagnostic in models with three or more fixed effects.

The OLS package-runtime tables compare six configurations: PyFixest MAP, PyFixest LSMR with no, diagonal, or factor-pair preconditioning, R `fixest`, and `FixedEffectModels.jl`:

Configuration	Package	Algorithm
<code>rust-map</code>	PyFixest	Vanilla Rust MAP, without acceleration.
<code>within-off</code>	PyFixest	LSMR without preconditioning.
<code>within-diagonal</code>	PyFixest	LSMR with diagonal preconditioning.
<code>within</code>	PyFixest	LSMR with factor-pair Schwarz preconditioning.
<code>fixest</code>	R <code>fixest</code>	Accelerated MAP with Irons-Tuck and other optimizations (Bergé et al. 2026).
<code>FEM.jl</code>	<code>FixedEffectModels.jl</code>	Diagonally preconditioned LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).

Tables report medians over the stated number of repetitions. All runs use an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1. We omit `reghdfe` (Correia 2014; 2017) because we lack a Stata license; its accelerated MAP algorithm is represented here by `fixest`.

7.1. Runtime Benchmarks

Package defaults are useful to users but do not impose equal accuracy because the implementations apply tolerances to different quantities. We report both package-default times and a comparison based on achieved coefficient and residual error.

The main OLS panels contain one million observations, one covariate, ten periods, and worker, firm, and year effects. One set of designs lowers the probability of moving, while the other holds that probability at one but increasingly assigns moves within groups of similar firms.

7.1.1. Worker Mobility

As mobility declines, our measure of connectivity for the worker-firm graph falls from 0.232 to 2.41×10^{-5} ; all configurations finish within 3.60 seconds in the first two designs. PyFixest MAP then slows sharply or reaches its iteration cap, and unpreconditioned LSMR also reaches the cap. Factor-pair LSMR falls from 0.554 seconds in the first design to 0.373 seconds in the last because the local worker-firm graphs become cheaper to factorize.

7.1.2. Sorting Among Movers

Although every worker moves between periods, increasing ρ concentrates moves within groups, and our measure of connectivity falls from 0.222 to 2.13×10^{-4} . PyFixest MAP rises from 0.411 seconds to 53.9 seconds before reaching its cap in the final design; unpreconditioned LSMR reaches its cap in the last four. Over the same designs, diagonal LSMR rises from 0.362 to 1.58 seconds, while factor-pair LSMR stays below one second and is faster than diagonal LSMR at the three smallest values of our measure of connectivity.

7.1.3. When Preconditioner Setup Dominates

The public fixest benchmark suite complements the gradual connectivity changes in the AKM designs by contrasting a dense, well-connected design with a sparse, nearly nested design (Bergé et al. 2026). Both contain 10 million observations, one covariate, and worker, firm, and year fixed effects.

Simple and difficult fixest designs (10 million observations, three fixed effects).

Design	$\lambda_2(\mathcal{L})$ (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
simple (well-connected)	0.622 (1.00)	2.35s	2.72s	2.16s	11.2s	2.52s	2.08s
difficult (near-nested)	8.35×10^{-8} (1.00)	326.2s	10.2s	capped (0/3)	4.63s	62.2s	27.0s

Note: Times are in seconds and cover each OLS regression from model setup through coefficient estimation. Each package uses its default settings. A preliminary run sets the number of measured runs: 20 if it takes less than 1 second, 7 if it takes 1 to 10 seconds, and 3 if it takes longer. When only k of n planned runs return an estimate, $t(\frac{k}{n})$ reports their median time t . A dash means that no timing is available. capped (0/ n) means that no run finishes before the iteration limit; failed (0/ n) marks a failure other than reaching that limit. Both designs have 10 million observations, one covariate, and worker, firm, and year fixed effects. Our measure of connectivity, λ_2 , is computed after iterative singleton removal on the component containing the most retained observations; its observation share appears in parentheses.

The dense design exposes the setup cost: five methods finish in 2.08 to 2.72 seconds, while factor-pair LSMR takes 11.2 seconds. Near nesting reverses the ranking. Factor-pair LSMR takes 4.63 seconds, compared with 10.2 seconds without preconditioning, 27.0 seconds for FEM.jl, 62.2 seconds for fixest, and 326.2 seconds for PyFixest MAP; diagonal LSMR reaches its default cap.

7.1.4. Iterations After Setup

To separate iteration counts from setup time, we use 100,000-observation versions of the same designs.

Iterations on the simple and difficult designs (100,000 observations).

Design	map (passes)	off	diagonal	additive
simple	14	37	16	14
difficult	8159	249	180	22

Note: The MAP column counts complete passes over all fixed-effect dimensions. The other columns count LSMR iterations: off uses no preconditioner, diagonal scales by fixed-effect group counts, and additive uses the factor-pair preconditioner. A MAP pass and an LSMR iteration perform different work, so their counts are not directly comparable. Each count is the median of three runs; each run removes the fixed effects from one outcome and one covariate.

Factor-pair LSMR needs 14 iterations on the simple design and 22 on the difficult one. The corresponding counts rise from 16 to 180 for diagonal LSMR and from 37 to 249 without preconditioning. The factor-pair iteration count is stable, but its setup still makes it unattractive for the simple one-shot regression.

7.1.5. Public Real-Data Benchmarks

The controlled designs isolate mobility, sorting, and near nesting, but empirical fixed-effect graphs also contain large differences in group size, thin links between dense groups, and many disconnected components. The public benchmark collection assembled by Correia (2017) contains eight such datasets and lets us check whether the controlled comparisons carry over to these less regular graphs.

Correia real-data benchmarks.

Dataset	$\lambda_2(\mathcal{L})$ (share)	PyFixest MAP	PyFixest LSMR factor-pair	fixest	FEM.jl
credit	0.227 (1.00)	0.193s	0.242s	0.154s	0.097s
soccer	0.666 (1.00)	0.026s	0.065s	0.014s	0.010s
enron	3.53×10^{-3} (0.99)	2.95s	0.402s	0.535s	0.317s
github	3.15×10^{-4} (0.32)	capped (0/3)	0.310s	1.43s	1.44s
patents	2.59×10^{-4} (0.95)	capped (0/3)	0.388s	1.05s	0.576s
workers	1.37×10^{-4} (0.63)	capped (0/3)	0.279s	2.00s	1.45s
schools	1.11×10^{-3} (1.00)	5.98s	0.266s	0.828s	0.487s
directors	2.56×10^{-4} (0.30)	capped (0/3)	0.327s	0.283s	0.462s

Note: Times are in seconds for OLS regressions using each package’s default settings, with three planned runs per cell. Each package applies its own rule for removing fixed-effect levels observed only once. If only k runs return an estimate, $t(\frac{k}{3})$ reports their median time t . capped (0/3) means that no run finishes before the 10,000-iteration limit. Our measure of connectivity, λ_2 , is computed after iterative singleton removal on the id1-id2 component containing the most retained observations; its observation share appears in parentheses.

On `credit` and `soccer`, PyFixest MAP takes 0.193 and 0.026 seconds, compared with 0.242 and 0.065 seconds for factor-pair LSMR, so the preconditioner’s setup cost does not pay on these easy datasets. Factor-pair LSMR is faster on the more difficult `enron` and `schools` graphs, taking 0.402 and 0.266 seconds compared with 2.95 and 5.98 seconds for PyFixest MAP. PyFixest MAP reaches its iteration cap on `github`, `patents`, `workers`, and `directors`, while factor-pair LSMR finishes in 0.279 to 0.388 seconds. Accelerated MAP can still be competitive, however; `fixest` takes 0.283 seconds on `directors`, slightly less than factor-pair LSMR’s 0.327 seconds.

Our measure of connectivity is a pairwise description rather than a rule for choosing a solver because runtime also depends on component size, factor dimensions, graph sparsity, acceleration, and the preconditioner’s setup cost.

7.2. Worker- and Firm-Specific Year Slopes

Applied work often allows a linear time trend to differ across observational units. We add one numeric year slope for every worker and firm to the simple and difficult designs and to all twelve AKM mobility and sorting designs, while retaining categorical year effects. In `fixest` formula syntax, the absorbed part is `indiv_id[year] +`

`firm_id[year] + year`. Each bracketed term includes a group intercept and a group-specific linear slope on the existing year variable, while the final term absorbs unrestricted aggregate year shocks.

Bracket syntax treats `year` as numeric; it does not create a categorical worker-year or firm-year effect. Every worker in these balanced panels contributes ten observations with year values 1 through 10, so the worker slopes do not create singleton groups. A firm slope requires observations from at least two years. Firms without that support are handled by each package’s usual identification and singleton rules. Worker-firm interactions are not included because they add categorical match effects rather than a slope on a numeric variable.

The PyFixest cells use the `within` backend because its other demeaning backends do not support varying slopes. We compare diagonal and factor-pair Schwarz preconditioning with R `fixest` and FEM.jl.

Worker- and firm-specific year slopes on the simple and difficult designs.

Design	PyFixest LSMR diagonal	PyFixest LSMR factor-pair Schwarz	fixest	FEM.jl
simple	-	-	-	-
difficult	-	-	-	-

Note: Times are in seconds for OLS regressions with 10 million observations and one covariate. Each cell has three planned runs after one unreported warm-up.

Worker- and firm-specific year slopes across AKM mobility designs.

Move probability δ	PyFixest LSMR diagonal	PyFixest LSMR factor-pair Schwarz	fixest	FEM.jl
1	0.675s	5.39s	20.7s	5.98s
0.5	2.46s	8.06s	39.4s	9.48s
0.05	capped (0/3)	16.8s	66.3s	22.2s
0.01	capped (0/3)	failed (0/3)	89.7s	20.9s
0.005	capped (0/3)	8.14s	105.0s	20.7s
0.001	capped (0/3)	8.55s	111.3s	15.9s

Worker- and firm-specific year slopes across AKM sorting designs.

Sorting strength ρ	PyFixest LSMR diagonal	PyFixest LSMR factor-pair Schwarz	fixest	FEM.jl
0	0.724s	3.46s	20.9s	5.90s
20	0.777s	8.22s	23.5s	5.78s
500	1.84s	8.72s	37.8s	5.77s
2000	2.31s	9.05s	46.9s	6.52s
10000	3.13s	11.3s	65.8s	7.30s
150000	4.02s	9.74s	63.9s	10.3s

Note: The AKM regressions have 1 million observations. Every regression in the three tables absorbs worker and firm intercepts, worker- and firm-specific numeric year slopes, and categorical year effects. PyFixest uses LSMR with diagonal or factor-pair Schwarz preconditioning through the within backend; R fixest and FEM.jl use their package defaults. If only k of the three runs return an estimate, $t(\frac{k}{3})$ reports the median time t . capped (0/3) means that no run finishes before its iteration limit, while failed (0/3) marks another failure.

7.3. Runtime and Achieved Precision

Solver comparisons also depend on how packages define convergence. MAP implementations compare changes in residuals or fixed-effect coefficients, while LSMR uses its estimated least-squares residual and first-order-condition error (Fong and Saunders 2011), so their numerical tolerances are not a common measure of accuracy. For example, within stops when either its relative residual or its scaled normal-equation error meets the requested threshold; we compute the latter as

$$\frac{\|A^T r\|_2}{\|A\|_F \|r\|_2}, \quad A = W^{\frac{1}{2}} D, \quad r = W^{\frac{1}{2}} (\mu - D\alpha). \quad (37)$$

Figure 7 places the methods on a common accuracy scale for mobility designs 1, 3, and 5. Every method receives the same singleton-pruned sample and a common 10,000-iteration cap. We time three fits at

each package-specific tolerance and compare their coefficients and residuals with factor-pair LSMR at tolerance 10^{-14} .

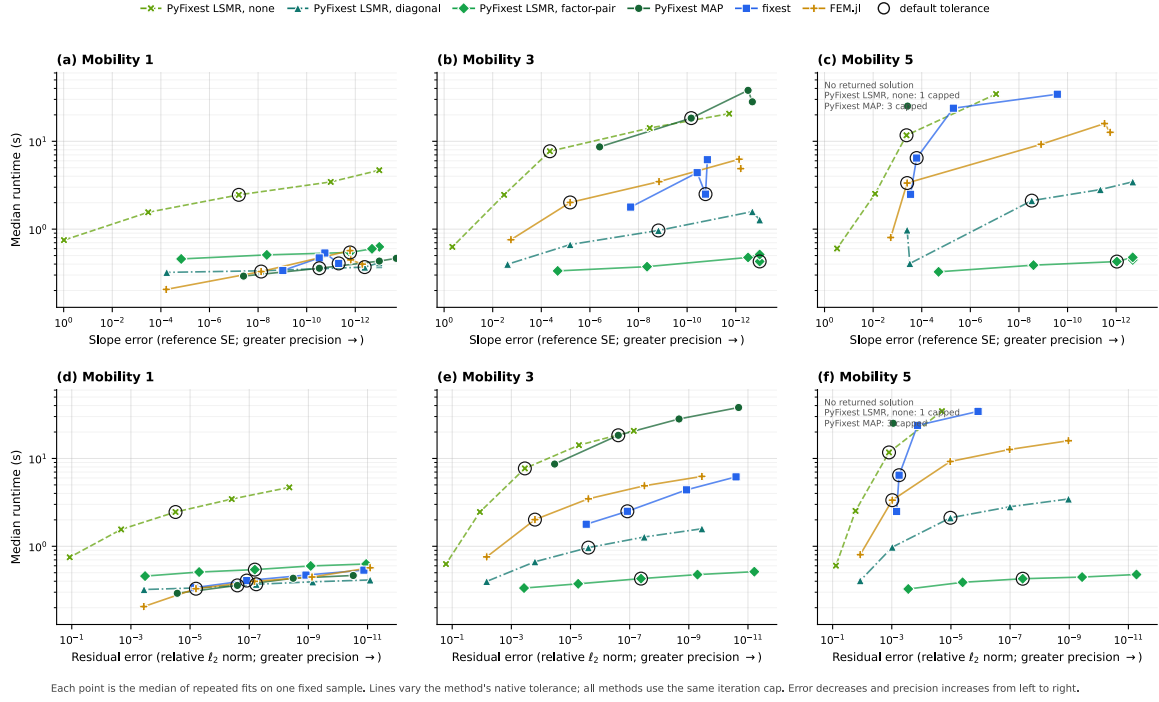


Figure 7: Elapsed time and achieved accuracy on three AKM mobility designs. Each marker reports the median elapsed time among the three planned fits that return an estimate on the same sample. Before timing, we remove fixed-effect levels observed only once and use the remaining observations for every method. Lines connect results for one method across requested tolerances. The top row measures coefficient error as $\frac{|\hat{\beta} - \hat{\beta}^*|}{SE(\hat{\beta}^*)}$, and the bottom row measures residual error as $\frac{\|r - r^*\|_2}{\|r^*\|_2}$; the reference coefficient $\hat{\beta}^*$ and residual r^* come from factor-pair LSMR at tolerance 10^{-14} . Error decreases from left to right, and a circle on each line marks the result obtained with the package's default tolerance. Every fit is limited to 10,000 iterations; settings for which none of the three fits returns an estimate are omitted and identified in the annotations.

7.4. Amortizing the Preconditioner

Matching achieved accuracy does not remove the factor-pair setup cost, but later regressions can reuse the pair blocks and factorizations when the observations, weights, and fixed-effect identifiers are unchanged.

7.4.1. Setup Cost Across Connectivity

We separate construction from LSMR iterations on the one-million-observation mobility designs, for specifications with worker-firm and worker-firm-year effects.

Factor-pair setup and LSMR times as worker mobility varies.

Move probability δ	$\lambda_2(\mathcal{L})$ (share)	2 FE setup	2 FE LSMR	3 FE setup	3 FE LSMR
1	0.232 (1.00)	0.109s	0.291s	0.113s	0.209s
0.5	0.0392 (1.00)	0.115s	0.255s	0.118s	0.234s
0.05	1.84×10^{-3} (0.87)	0.032s	0.135s	0.034s	0.233s
0.01	5.33×10^{-4} (0.61)	0.027s	0.124s	0.031s	0.216s
0.005	5.51×10^{-5} (0.52)	0.025s	0.093s	0.031s	0.193s
0.001	2.41×10^{-5} (0.29)	0.028s	0.045s	0.032s	0.149s

Note: Times are in seconds. For each design and specification, we plan five runs with 1 million observations and an LSMR tolerance of 10^{-12} . The table gives the median setup time and time spent in LSMR among the runs that finish; if only k finish, $t(\frac{k}{5})$ reports their median time t . The two-fixed-effect specification absorbs worker and firm effects; the three-fixed-effect specification also absorbs year effects. Our measure of connectivity, λ_2 , is computed after iterative singleton removal on the component containing the most retained observations; its observation share appears in parentheses.

From the highest- to the lowest-mobility design, setup falls from 0.109 to 0.028 seconds with two fixed effects and from 0.113 to 0.032 seconds with three; time spent in LSMR also falls. The low-mobility graph has fewer cross-firm links to store and factorize, which explains the declining factor-pair runtime in Figure 1.

7.4.2. Ten Regressions on the Same Fixed Effects

Empirical projects often estimate several specifications on one sample and set of fixed effects. We time ten sequential regressions on both one-million-observation `fixest` designs, comparing diagonal preconditioning with factor-pair preconditioning that is rebuilt for each regression or retained for all ten.

Ten regressions with rebuilt and cached preconditioners.

Design	Policy	Setup	LSMR	Total	Speedup
simple	Diagonal	0.048s	0.553s	0.601s	1.0x
	Additive, rebuilt	2.80s	1.51s	4.29s	0.1x
	Additive, cached	0.281s	1.51s	1.79s	0.3x
difficult	Diagonal	0.049s	43.6s	43.6s	1.0x
	Additive, rebuilt	0.379s	1.20s	1.58s	27.5x
	Additive, cached	0.040s	1.26s	1.30s	33.6x

Note: For each policy, we add the setup and LSMR times across ten sequential regressions and take the median across three planned repetitions. If only k repetitions finish, $t(\frac{k}{3})$ reports their median time t , and the speedup is omitted. Times are in seconds. Each design has 1 million observations and worker, firm, and year fixed effects. Each regression removes the fixed effects from the common outcome and one of ten covariates at an LSMR tolerance of 10^{-12} . For complete cells, speedup divides the diagonal total time by the reported total time. `Additive, rebuilt` constructs a new factor-pair preconditioner for each regression; `Additive, cached` reuses one preconditioner for all ten.

Reuse lowers factor-pair time on the simple design from 4.29 to 1.79 seconds, but diagonal preconditioning still takes only 0.601 seconds. On the difficult design, the ranking is decisive: ten regressions take 43.6 seconds with diagonal preconditioning, 1.58 seconds when the factor-pair preconditioner is rebuilt, and 1.30 seconds when it is retained.

7.5. Poisson and Other GLMs

Reuse also matters within a single nonlinear fit because Poisson fixed-effect models repeat the absorption step at every iteratively reweighted least-squares (IRLS) iteration (Stammann 2018). Their fixed-effect identifiers remain fixed while the weights and working response change, so `ppmlhdfc` (Correia et al. 2020) and related estimators repeatedly residualize the data with the same identifiers and accumulate this cost over the fit.

PyFixest currently retains the preconditioner from the first weighted demeaning call as the IRLS weights change. The benchmark reports that policy; it does not isolate reuse as the cause of the runtime difference.

The comparison uses the one-million-observation simple and difficult `fixest` designs (Bergé et al. 2026). It includes R `fixest::fepois`, `GLFixedEffectModels.jl`, and PyFixest `fepois` with MAP or factor-pair LSMR. Each implementation receives the same 100-iteration IRLS limit and applies its default separation rule.

Poisson benchmarks (1 million observations, one covariate).

Design	FE	PyFixest MAP	fixest	GLFEM.jl	PyFixest LSMR factor-pair
simple (well-connected)	3	7.95s	4.63s	5.71s	9.74s
difficult (near-nested)	3	capped (0/3)	439.3s	129.8s	5.52s

Note: Times are in seconds for Poisson fixed-effect regressions with 1 million observations, one covariate, and worker, firm, and year fixed effects. `fixest` is R `fixest::fepois`; `rust-map` and `within` use PyFixest `fepois`; and `GLFEM.jl` is `GLFixedEffectModels.jl`. The `within` configuration reuses the first factor-pair preconditioner as the weights change across iteratively reweighted least squares steps. Each package applies its default rule for removing separated observations. If only k of the three planned fits return an estimate, $t(\frac{k}{3})$ gives their median time t . `capped (0/3)` means that no fit finishes before either the package’s inner demeaning limit or the common limit of 100 iteratively reweighted least squares steps; `failed (0/3)` marks another failure.

All four implementations finish the simple design in less than ten seconds. On the difficult design, PyFixest MAP reaches its iteration cap, `GLFEM.jl` takes 129.8 seconds, `fixest` takes 439.3 seconds, and factor-pair LSMR takes 5.52 seconds. Repeated absorption magnifies the cost of slow propagation across the worker-firm graph.

8. Software

The benchmarks use the open-source `within` solver (within Developers 2026), whose computational core is written in Rust and exposed through Python and R APIs.

Interface	Package	Registry	Install
Rust	<code>within</code>	crates.io	<code>cargo add within</code>
Python	<code>within-py</code>	PyPI	<code>pip install within-py</code>
R	<code>withinr</code>	py-econometrics R-universe	<code>install.packages("withinr", repos = "https://py-econometrics.r-universe.dev")</code>

PyFixest users can select the method without constructing the Gramian or its pair blocks (PyFixest Developers 2026):

```
import pyfixest as pf

fit = pf.feols(
    "y ~ x1 + x2 | worker + firm",
    data=df,
    demeaner=pf.LsmrDemeaner(),
)
```

The lower-level Python interface exposes the FWL operation, with `worker_code` and `firm_code` supplied as zero-based integer codes:

```
import numpy as np
from within import solve_batch
```

```
categories = np.asfortranarray(
    df[["worker_code", "firm_code"]].to_numpy(dtype=np.uint32)
)
variables = np.asfortranarray(df[["y", "x1", "x2"]].to_numpy())
res = solve_batch(categories, variables)
y_tilde, X_tilde = res.demeaned[:, 0], res.demeaned[:, 1:]
beta_hat = np.linalg.lstsq(X_tilde, y_tilde, rcond=None)[0]
```

`solve_batch` residualizes all three columns with one factor-pair preconditioner, after which the last line runs the low-dimensional FWL regression from Section 2.

9. Conclusion

MAP's low cost per pass makes it effective for many well-connected fixed-effect models, but successive mean updates move information slowly when mobility is sparse, sorting is strong, or one effect is nearly nested in another.

Factor-pair preconditioning uses observed co-occurrences to correct worker-firm and other pair blocks jointly, changing the numerical update while preserving the original least-squares target. Its setup cost can outweigh the reduction in iterations on easy graphs, but on difficult graphs it can avoid hundreds of MAP passes or LSMR iterations; sparse pair graphs can also be cheaper to factorize than dense ones.

The setup cost is most likely to be worthwhile when our measure of connectivity for a factor pair is small or the same fixed effects are absorbed repeatedly. Specification searches can reuse one preconditioner across outcomes and covariates, while IRLS estimators call the demeaning routine at every outer iteration. In both cases, reuse spreads the setup cost across several residualizations. The measure remains a pairwise connectivity diagnostic rather than a solver-selection rule, but a small value signals the weak connectivity that can slow MAP.

Appendix: Memory Use

MAP stores the current residuals and group sums, whereas factor-pair LSMR also retains match structures and local factorizations. We compare peak resident memory for the two Rust implementations inside PyFixest, using isolated processes so the surrounding regression code is held fixed.

Peak physical memory use (three fixed effects, one covariate).

Design	$\lambda_2(\mathcal{L})$ (share)	PyFixest MAP	PyFixest LSMR factor-pair
<i>100K observations</i>			
simple (well-connected)	0.622 (1.00)	354 MiB	411 MiB
difficult (near-nested)	6.51×10^{-4} (1.00)	357 MiB	383 MiB
<i>1M observations</i>			
simple (well-connected)	0.622 (1.00)	758 MiB	1,000 MiB
difficult (near-nested)	8.35×10^{-6} (1.00)	775 MiB	953 MiB

Note: For each regression, we record the largest amount of physical memory used by an isolated Python process, in MiB (2^{20} bytes). The table compares PyFixest OLS regressions using MAP or factor-pair LSMR. Each regression has one covariate and worker, firm, and year fixed effects. The results cover the simple and difficult designs at 100,000 and 1 million observations. Our measure of connectivity, λ_2 , is computed after iterative singleton removal on the component containing the most retained observations; its observation share appears in parentheses.

Factor-pair LSMR uses slightly more memory: 26-57 MiB at 100K observations and 178-242 MiB at 1M; the additional storage holds the pairwise co-occurrences, partition weights, and approximate Cholesky factors.

Appendix A: Details of the Factor-Pair Schwarz Preconditioner

A.1 Two-Factor Block Inverse

For the two-factor block

$$G_2 = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \quad (38)$$

let

$$S = G_{FF} - C_{(WF)'} G_{WW}^{-1} C_{WF}. \quad (39)$$

Standard block inversion gives

$$G_2^{-1} = \begin{pmatrix} G_{WW}^{-1} + G_{WW}^{-1} C_{WF} S^{-1} C_{(WF)'} G_{WW}^{-1} & -G_{WW}^{-1} C_{WF} S^{-1} \\ -S^{-1} C_{(WF)'} G_{WW}^{-1} & S^{-1} \end{pmatrix}. \quad (40)$$

A.2 Algorithm

Algorithm 1 gives the implementation corresponding to the construction summarized in Figure 6.

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q absorbed dimensions.
- Diagonal weights W and a local solver configuration.
- Krylov residual r in coefficient space.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks G_{qq} , G_{rr} and the weighted cross-tabulation C_{qr} .
- Split the induced bipartite graph into connected components and create one Schwarz subdomain s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, form $L_s = T_s G_s T_s$. If p_s is the number of local levels, set $\Pi_s = I_{p_s} - \mathbf{1} \frac{1'}{p_s}$; multiplying by Π_s subtracts the component mean.
- Eliminate the larger factor block, leaving a reduced system on the smaller block. Solve a small reduced system by dense Cholesky. For a large system, approximate the fill-in cliques by sampling and apply randomized approximate Cholesky.

Krylov application

- Initialize $z = 0$.
- For each subdomain s , form $h_s = \Omega_s R_s r$ and $b_s = \Pi_s T_s h_s$.
- Apply the stored local solver to b_s to obtain v_s , then set $u_s = T_s v_s$.
- Accumulate $z \leftarrow z + R_s' \Omega_s u_s$.
- Return $z = M^{-1} r$.

References

- Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. "High Wage Workers and High Wage Firms." *Econometrica* 67 (2): 251–333. <https://doi.org/10.1111/1468-0262.00020>.
- Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. "Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids." *Inverse Problems* 30 (7): 75009. <https://doi.org/10.1088/0266-5611/30/7/075009>.
- Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm*. Technical Report Nos. 2018–13.
- Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. "Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R." <https://doi.org/10.48550/arXiv.2601.21749>.
- Bonhomme, Stéphane, Kerstin Holzheu, Thibaut Lamadon, Elena Manresa, Magne Mogstad, and Bradley Setzler. 2023. "How Much Should We Trust Estimates of Firm Effects and Worker Sorting?." *Journal of Labor Economics* 41 (2): 291–322. <https://doi.org/10.1086/720009>.
- Chung, Fan R. K. 1997. *Spectral Graph Theory*. Vol. 92, pt. 92. CBMS Regional Conference Series in Mathematics. American Mathematical Society.
- Correia, Sergio. 2014. "Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects." <https://github.com/sergiocorreia/reghdfe>.
- Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator*. Technical report. <https://hdl.handle.net/10.2139/ssrn.3129010>.
- Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. <https://doi.org/10.1177/1536867X20909691>.
- FixedEffectModels.jl Developers. 2026. "Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables." <https://github.com/FixedEffects/FixedEffectModels.jl>.
- Fong, David Chin-Lung, and Michael Saunders. 2011. "LSMR: An Iterative Algorithm for Sparse Least-Squares Problems." *SIAM Journal on Scientific Computing* 33 (5): 2950–71. <https://doi.org/10.1137/10079687X>.
- Frisch, Ragnar, and Frederick V. Waugh. 1933. "Partial Time Regressions as Compared with Individual Trends." *Econometrica* 1 (4): 387–401. <https://doi.org/10.2307/1907330>.
- Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. "AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization." *SIAM Journal on Scientific Computing*, <https://rasmuskyng.com/papers/GKS25.pdf>.
- Gaure, Simen. 2013. "OLS with Multiple High Dimensional Category Variables." *Computational Statistics & Data Analysis* 66 : 8–18. <https://doi.org/10.1016/j.csda.2013.03.024>.
- Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields*. Technical report.
- Guimaraes, Paulo, and Pedro Portugal. 2010. "A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects." *The Stata Journal* 10 (4): 628–49. <https://doi.org/10.1177/1536867X1101000406>.

- Irons, Bruce M., and Richard C. Tuck. 1969. "A Version of the Aitken Accelerator for Computer Iteration." *International Journal for Numerical Methods in Engineering* 1 (3): 275–77. <https://doi.org/10.1002/nme.1620010306>.
- Jochmans, Koen, and Martin Weidner. 2019. "Fixed-Effect Regressions on Network Data." *Econometrica* 87 (5): 1543–60. <https://doi.org/10.3982/ECTA14605>.
- Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010. <https://doi.org/10.1080/01621459.1963.10480682>.
- PyFixest Developers. 2026. "Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python." <https://github.com/py-econometrics/pyfixest>.
- Spielman, Daniel A., and Shang-Hua Teng. 2014. "Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems." *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. <https://doi.org/10.1137/090771430>.
- Stammann, Amrei. 2018. "Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects." <https://doi.org/10.48550/arXiv.1707.01815>.
- Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. <https://doi.org/10.1007/b137868>.
- within Developers. 2026. "Within: High-Performance Fixed-Effects Solver for Econometric Panel Data." <https://github.com/py-econometrics/within>.
- Xu, Jinchao. 1992. "Iterative Methods by Space Decomposition and Subspace Correction." *SIAM Review* 34 (4): 581–613. <https://doi.org/10.1137/1034116>.
- Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2025. "Flexible Modified LSMR for Least Squares Problems." *Numerical Algebra, Control and Optimization* 15 (4): 1321–39. <https://doi.org/10.3934/naco.2025019>.